

Sampling plan for Kolmogorov inverse problems

Что уже реализовано, как этим пользоваться, что добавить для conditional sampling

Что уже сделано

Датасет

Основной датасет для дальнейших inverse experiments — velocity snapshots Kolmogorov flow:

$$u = (u_x, u_y), \quad [N, 2, 64, 64].$$

Данные генерируются в kolmogorov_dataset/. Симуляция идет в vorticity form на periodic grid. В текущем варианте поле считается на 256×256 , затем velocity усредняется до 64×64 . В .npz сохраняются raw values, без нормализации.

Что лежит в одном chunk:

- images: velocity fields $[N, 2, H, W]$;
- trajectory_id, snapshot_index, step: откуда взят snapshot;
- viscosity, drag, forcing_amp: параметры симуляции;
- split: train / val / test.

Обучение prior

Основной prior — VP-SDE score model из score_training/. Модель предсказывает noise ε из зашумленного поля:

$$x_t = \mu_t x_0 + \sigma_t \varepsilon, \quad \mu_t = \cos(\omega t), \quad \sigma_t = \sqrt{1 - \mu_t^2}.$$

Вход модели:

$$[x_t, \sin x, \cos x, \sin y, \cos y].$$

Координатные Fourier channels добавляются как clean conditioning channels: они не шумятся и не участвуют в loss. UNet — diffusers.UNet2DModel с circular padding.

Нормализация считается только по train split:

$$x_{\text{norm}} = \frac{x_{\text{raw}} - \text{mean}}{\text{std}}.$$

mean и std сохраняются в checkpoint. Все samplers должны работать в normalized space. Метрики и картинки нужно считать в raw space.

Unconditional sampling

В score_training/sde.py уже есть deterministic VP sampler. На каждом шаге:

$$\hat{x}_0 = \frac{x_t - \sigma_t \varepsilon_\theta(x_t, t)}{\mu_t}, \quad x_{t_{\text{next}}} = \mu_{t_{\text{next}}} \hat{x}_0 + \sigma_{t_{\text{next}}} \varepsilon_\theta(x_t, t).$$

Conditional samplers используют тот же reverse process. Сначала модель предсказывает $\varepsilon_\theta(x_t, t)$, затем из него считается $\hat{x}_0$. После этого sampler добавляет информацию из observation y : либо меняет $\hat{x}_0$, либо делает correction для x_t , либо склеивает known/unknown regions.

Observation operators

Для сравнения методов будут использоваться четыре фиксированных оператора наблюдения. Все они применяются к двум velocity channels одинаково.

- Regular sparse grid. Известны значения u_x, u_y в узлах регулярной сетки с шагом 4 по обеим координатам: $M_{ij} = 1$ для $i \bmod 4 = 0, j \bmod 4 = 0$. Observation имеет тот же spatial size 64×64 , но значения вне mask не используются.
- Central box mask. Известно полное velocity field внутри центрального окна 32×32 :

$$i, j \in \{16, \dots, 47\}.$$

Вне окна поле неизвестно.

- Average downsample. Observation получается average pooling:

$$A : [2, 64, 64] \rightarrow [2, 16, 16],$$

kernel 4×4 , stride 4.

- Periodic Gaussian blur. Observation получается circular convolution с Gaussian kernel, $\sigma = 2$ grid cells. После blur добавляется Gaussian noise с фиксированным σ_y .

Sampling methods

DDNM

Используется для линейных операторов, где есть $A^\dagger$. Самый простой первый case — mask/inpainting.

На каждом reverse step:

$$\hat{x}_0 = \frac{x_t - \sigma_t \varepsilon_\theta(x_t, t)}{\mu_t}.$$

Затем заменить observed part:

$$\hat{x}_0^{\text{corr}} = A^\dagger y + (I - A^\dagger A) \hat{x}_0.$$

После этого сделать обычный VP reverse step, но уже из $\hat{x}_0^{\text{corr}}$.

Для mask оператор $A(x) = M \odot x$, а $A^\dagger(y) = M \odot y$. Поэтому correction упрощается до замены known pixels:

$$\hat{x}_0^{\text{corr}} = y_{\text{known}} + (1 - M) \odot \hat{x}_0.$$

Для noisy observations вместо жесткой замены лучше использовать soft update:

$$\hat{x}_0^{\text{corr}} = \hat{x}_0 + \lambda_t A^\dagger(y - A \hat{x}_0).$$

DPS

DPS работает для любого differentiable A . Он не требует $A^\dagger$, но требует gradient через модель.

На каждом шаге x_t должен быть differentiable. Модель дает $\hat{x}_0(x_t, t)$, после чего считается measurement loss:

$$L_y = \|A(\hat{x}_0) - y\|_2^2.$$

Gradient по текущему noisy state:

$$g = \nabla_{x_t} L_y$$

используется как correction:

$$x_t^{\text{corr}} = x_t - s_t g.$$

Дальше reverse step идет из corrected state.

Веса модели не обновляются. Optimizer не нужен. Нужно только autograd по x_t .

Параметры:

- guidance_scale: общий масштаб gradient step;
- measurement_sigma: шум observation;
- guidance_start, guidance_end: диапазон t , где включается guidance;
- gradient clipping или normalization для устойчивости.

RePaint

RePaint использовать только для mask. Для blur/downsample он не подходит.

Идея: known region каждый раз заменяется observation, зашумленным до текущего уровня t . Unknown region генерируется моделью.

Known part строится через forward noising observation:

$$x_t^{\text{known}} = \mu_t y + \sigma_t \varepsilon.$$

Unknown part берется из prior step модели. Затем они склеиваются по mask:

$$x_t = M \odot x_t^{\text{known}} + (1 - M) \odot x_t^{\text{generated}}.$$

Jumps добавляются поверх этого процесса: sampler иногда переходит назад к более шумному состоянию и повторяет несколько reverse steps, чтобы согласовать границу между observed и generated regions.

DDRM

DDRM нужен для линейных операторов, которые удобно диагонализировать. Для нас это blur и часть downsample/super-resolution experiments. Делать его после DDNM и DPS.

Идея: работать в basis, где A раскладывается по singular values:

$$A = U \Sigma V^\top.$$

Для каждого spectral component отдельно смешивается prior prediction и measurement.

Для periodic blur удобно использовать Fourier basis. Blur тогда становится умножением на transfer function $H(k)$. Correction считается по каждому k отдельно, с учетом $H(k)$, текущего diffusion noise и measurement noise σ_y .

Methods vs observations

Method	Sparse grid	Box mask	Downsample	Blur
DDNM	yes	yes	yes	yes
DPS	yes	yes	yes	yes
RePaint	yes	yes	no	no
DDRM	not primary	not primary	yes	yes

DDNM and RePaint are most natural for masks. DPS is the general differentiable baseline and can be run for all four operators. DDRM is reserved for operators with a convenient spectral representation: downsample and blur.

Physics-informed sampling

Для Kolmogorov velocity fields основное физическое ограничение:

$$\nabla \cdot u = 0.$$

Его можно добавлять к sampling без переобучения модели. Базовый вариант — Helmholtz projection в Fourier space:

$$\hat{u}_{\text{proj}}(k) = \hat{u}(k) - k \frac{k \cdot \hat{u}(k)}{\|k\|^2}, \quad k \neq 0.$$

Она удаляет compressible component и оставляет divergence-free part поля. Projection нужно применять в normalized model space или в raw space согласованно с тем, где делается correction. Для финальных метрик поле все равно переводится в raw space.

DDNM + physics

В DDM physics удобно применять к $\hat{x}_0^{\text{corr}}$. Сначала делается data-consistency correction:

$$\hat{x}_0^{\text{corr}} = A^\dagger y + (I - A^\dagger A) \hat{x}_0,$$

после этого к corrected estimate применяется Helmholtz projection:

$$\hat{x}_0^{\text{phys}} = P_{\text{divfree}}(\hat{x}_0^{\text{corr}}).$$

Дальше VP reverse step использует $\hat{x}_0^{\text{phys}}$.

Для mask observations есть важный trade-off: hard projection может немного изменить known pixels. Если нужно строго сохранить observation, можно делать projection только на unknown part или после projection повторно вставлять known pixels.

DPS + physics

В DPS physics можно добавить как дополнительный guidance term:

$$L = L_y + \lambda_{\text{div}} L_{\text{div}}, \quad L_{\text{div}} = \|\nabla \cdot \hat{x}_0\|_2^2.$$

Тогда gradient correction считается по общей функции L :

$$x_t^{\text{corr}} = x_t - s_t \nabla_{x_t} L.$$

Это soft physics guidance. Его удобно использовать вместе с differentiable operators, потому что DPS уже требует autograd по x_t .

Альтернативный вариант — не добавлять L_{div} , а после DPS correction проецировать $\hat{x}_0$ через Helmholtz projection. Это проще и стабильнее, но может конфликтовать с observation consistency для mask.

RePaint + physics

В RePaint known region задается observation, unknown region генерируется prior-ом. Поэтому physics projection лучше применять к generated/unknown part, а known part затем снова вставлять по mask:

$$x = M \odot x_{\text{known}} + (1 - M) \odot P_{\text{divfree}}(x_{\text{generated}}).$$

Если применить projection ко всему полю сразу, known pixels могут измениться. Это плохо для inpainting setting, где observed region должен оставаться фиксированным.

DDRM + physics

В DDRM data update происходит в spectral/SVD basis. Physics projection тоже удобно делать в Fourier space, поэтому для blur этот вариант естественный: после spectral DDRM update можно применить Helmholtz projection к velocity field.

Для downsample projection тоже возможна, но порядок важен. Практичный вариант: сначала выполнить DDRM measurement update, затем Helmholtz projection, затем следующий reverse step. Если projection заметно ухудшает consistency с low-resolution observation, можно добавить повторную мягкую correction по $A(\hat{x}_0) - y$.

Evaluation pipeline

Evaluation должен быть одинаковым для всех методов. Для held-out field u_{true} строится observation

$$y = A(u_{\text{true}}) + \eta.$$

Sampler получает один и тот же checkpoint, один и тот же operator A , один и тот же noise level и фиксированный seed. Внутри sampler все вычисления идут в normalized space. После sampling reconstruction переводится обратно в raw velocity space, и уже там считаются метрики и строятся plots.

Для каждого запуска сохраняется одна строка в CSV. Для небольшой подвыборки сохраняются PNG visualizations: ground truth, observation, reconstruction, error map и vorticity.

Минимальный набор experiments:

- regular sparse grid, stride 4;
- central box mask, 32×32 ;
- downsample: $64 \times 64 \rightarrow 16 \times 16$;
- Gaussian blur, $\sigma = 2$, with Gaussian observation noise.

Для debug run достаточно $N = 16$, one seed. Для финальной таблицы: $N = 256$, seeds $\{0, 1, 2\}$.

Metrics

Все метрики считаются в raw velocity space.

- Relative L2:

$$\frac{\|\hat{u} - u_{\text{true}}\|_2}{\|u_{\text{true}}\|_2}.$$

- RMSE:

$$\sqrt{\frac{1}{2HW} \|\hat{u} - u_{\text{true}}\|_2^2}.$$

- Measurement consistency:

$$\frac{\|A(\hat{u}) - y\|_2}{\|y\|_2}.$$

- Divergence:

$$\frac{\|\nabla \cdot \hat{u}\|_2}{\|\hat{u}\|_2}.$$

- Vorticity RMSE. Сначала считать

$$\omega = \partial_x u_y - \partial_y u_x,$$

затем RMSE между $\omega(\hat{u})$ и $\omega(u_{\text{true}})$.

CSV columns:

sample_id, seed, method, operator, noise_sigma,
rel_l2, rmse, measurement_error, divergence, vorticity_rmse